

CSC310 Data Structures and Algorithm Analysis (Dr. Ng)

Programming Assignment 4

Extend the 3 by 3 “Tic-Tac-Toe” program in assignment #1 to allow up to 8 players and any board size (M by N) including the following features:

1. The program would allow 3 and up to 8 human players. Each game is played by the SAME number of players once it starts.
2. The program will capture player's full names (only the first name and last name separated by space(s)). A valid player's name consists of only alphabet letters. Players' names are entered one player at a time per line.
3. The first player's piece is “z”, the second player's piece is “y”, and so on. The eighth player's piece is “s”. All pieces are lower case alphabets. The players keep their pieces throughout the game.
4. The players can specify the size of the board (up to 11 by 14) for EACH game. The number of rows and columns are input on separate prompts. Each game might have a different board size. The minimum size of the board is 4 by 4.
5. Player with piece “x” starts the first game. The order of players will be in the round robin sequence “z”, “y”, “x”, “w”, “v”, “u”, “t”, “s” and then back to “z”. Depending on the number of players, the round robin sequence might wrap around earlier before it reaches “s”.
6. The initial game board (example: 4 X 6) should exactly like the one below:

```
Please enter the dimension of the board.
Enter the number of rows    -> 4
Enter the number of columns -> 12

  1  2  3  4  5  6  7  8  9 10 11 12
  --
A |  |  |  |  |  |  |  |  |  |  |  | A
  --
B |  |  |  |  |  |  |  |  |  |  |  | B
  --
C |  |  |  |  |  |  |  |  |  |  |  | C
  --
D |  |  |  |  |  |  |  |  |  |  |  | D
  --
  1  2  3  4  5  6  7  8  9 10 11 12
```

7. All players' names should be displayed in the format of starting with a capitalized letter and followed by all lower-case letters.
8. Only the first name of players is to be used in subsequent prompt messages (#9 below). However, the full name will be used in displaying game statistics (#14 below)
9. The program will prompt user's move accordingly using only the first name of the player.
10. A player's move is specified as row character letter (A..K, or a..k), followed by column number (1..14), a single entry, e.g. D5, K12, b4, d6, etc. The row character letter can be upper case or lower case.
11. The program would redraw the board ONCE after each valid move.

12. A player wins the game if he has three of his pieces consecutively horizontally, vertically or diagonally anywhere in the board, which is defined as a winning configuration.
13. If one player wins, pieces in **ALL possible winning configurations** will be displayed on the board in **upper case**. All other pieces remain unchanged.
14. At the end of each game, the program would **display the total number of games played** and **the number of games each player wins, loses and draws in a table form**. The full names of the players are displayed **aligned right, in the order of players' names entered at the beginning of the game**. The number in the statistics also aligned right (assuming a maximum of 1000 games can be played any time). An example of such statistics is as below:

```
Total game played = 1
```

	WIN	LOSS	DRAW
John Doe	0	1	0
Jane Doe	0	1	0
Buster Brown	0	1	0
Mighty Warrior	1	0	0

15. If a game ends with someone winning, the next game starts with the player AFTER the winner of the previous game in the round robin sequence as described in #5, i.e. the winner of the previous game will be the last one in the next game.
16. If a game ends in a draw, the next game starts with the players after the previous game's starter in the round robin sequence as described in #5.
17. The program would allow multiple games to be played until the players select to quit at the end of any game.

In addition, **your program should account for all possible user input errors**.

You should use C++ for this assignment. You cannot call any library functions except those dealing with I/O and string handling functions. Sophisticated graphic display is NOT necessary, and it wouldn't get you extra points. All that is required is a neat and reasonable user-friendly user interface.

Your program should execute interactively on your computer. Please test your program thoroughly and make sure it is free of errors. Points will be deducted for each error that I discovered for you.

Please also pay attention to program design and follow all the good programming practice. I reserve the right to deduct points based on these criteria.

Submit: Program design
Hierarchical diagrams/ Structure charts
Procedure specification
Program listing
Testing cases with Justification
Results of test cases

TIC-TAC-TOE Game Extension:

Let define general TicTacToe game as $TTT(p, m, n, k)$ as:

$TTT(p, m, n, k)$ = p players, m by n board, and any k -consecutive pieces to win.

Assignment 1 is $TTT(2,3,3,3)$:

Assignment 2 is $TTT(p, m, n, 3)$ where $3 \leq m \leq 12$, $3 \leq n \leq 14$

For those who finish this assignment early, you can extend this assignment in the following ways:

- Extend assignment 2 to $TTT(p, m, n, k)$, where $k = 4,5,6,\dots$
- Extend Assignment 1 to a TicTacToe game where each square is a basic $TTT(2,3,3,3)$ game.
- Extend 2-dimensional TicTacToe game to 3-dimensional game.

Extension to other similar board games:

For those who are bored with TicTacToe game,

- Modify your program to play the standard 7 by 7 two players “connect-4” board game.
- Modify your program to play the standard M by N two players “connect- X ” board game where the value of M , N and X are input by the players.
- Modify your program to play the standard two players, 8 by 8 “Reversi” board game.

Reflection Questions:

- What are the commonalities of all these board games?
- Write a generic class that captures the commonality of board games?